Compte rendu TP 2

1. Architecture et fichiers de configuration

J'ai construit une PKI à deux niveaux avec OpenSSL 3 : une AC racine (P-384, 10 ans, hors ligne en production), une AC intermédiaire (P-256, 5 ans) limitée au domaine tp.local, puis un certificat de serveur (www.tp.local) et deux certificats de client (alice@tp.local, bob@tp.local) valables 90 jours. Les deux fichiers racine.cnf et inter.cnf sont ceux de l'annexe B, avec deux ajouts : unique_subject = no (nécessaire pour renouveler un certificat avec le même sujet, utile au TP 3) et une section ext_client_bob sans SAN fixe, pour que Bob ne porte pas le SAN alice@tp.local défini dans ext_client.

Rôle de chaque ligne des sections d'extensions

Ligne	Rôle
basicConstraints = critical,CA:TRUE,pathlen:0 (ext_inter)	Le certificat est celui d'une AC, qui ne peut pas créer d'AC en dessous d'elle (pathlen:0). critical : un client qui ne comprend pas cette ligne doit rejeter le certificat.
keyUsage = critical,keyCertSign,cRLSign (ext_inter)	La clé sert uniquement à signer des certificats et des CRL.
nameConstraints = critical,permitted;DNS:.tp.local,permitted; DNS:tp.local,permitted;email:tp.local	L'intermédiaire ne peut valablement émettre que pour les sous-domaines de tp.local (avec le point), pour tp.local lui-même et pour les adresses e-mail de ce domaine.
basicConstraints = critical,CA:FALSE (ext_serveur, ext_client)	Certificat « feuille » : il ne peut signer aucun autre certificat.
keyUsage = critical,digitalSignature	La clé sert à signer (authentification TLS), pas à chiffrer ni à signer des certificats.
extendedKeyUsage = serverAuth / clientAuth	Usage autorisé : serveur TLS ou client TLS. Un certificat client est refusé comme serveur, et inversement.
subjectAltName = email:alice@tp.local (ext_client)	Identité de l'utilisateur dans le certificat, ici fixée dans la configuration fournie.
subjectKeyIdentifier = hash	Identifiant du certificat : empreinte de sa clé publique.
authorityKeyIdentifier = keyid	Identifiant de la clé de l'émetteur, pour reconstruire la chaîne.
copy_extensions = copy (inter) / none (racine)	L'intermédiaire recopie dans le certificat le SAN demandé dans la CSR ; la racine ignore les extensions de la CSR et n'applique que les siennes.
default_crl_days = 7 (inter)	La CRL de l'intermédiaire expire 7 jours après sa publication : elle doit être republiée chaque semaine.

2. AC racine

Élément	Valeur observée
Émetteur	C=BF, O=Universite TP, CN=Racine TP
Sujet	C=BF, O=Universite TP, CN=Racine TP (identique à l'émetteur)
Validité	27 oct. 2026 → 24 oct. 2036 (10 ans)
Algorithme de clé	id-ecPublicKey, courbe secp384r1 (P-384), signature ecdsa-with-SHA384

Élément	Valeur observée
Extensions critiques	Basic Constraints : CA:TRUE, pathlen:1 ; Key Usage : Certificate Sign, CRL Sign

Pourquoi l'émetteur et le sujet sont-ils identiques ? La racine est auto-signée : personne au-dessus d'elle ne peut la signer. Elle est le point de départ de la confiance et doit être installée directement dans les magasins de confiance. pathlen:1 autorise une seule AC intermédiaire sous elle.

3. AC intermédiaire

L'intermédiaire génère sa clé P-256 et une CSR ; la racine la signe avec openssl ca -config racine.cnf -extensions ext_inter -days 1825. Dans émetteur Racine TP, sujet Inter TP, validité du 30 oct. 2026 au 29 oct. 2031, CA:TRUE, pathlen:0 et la contrainte de nom critique avec DNS:.tp.local, DNS:tp.local, email:tp.local. L'intermédiaire a été re-signée avec la même clé pour appliquer les contraintes de l'annexe B ; les certificats déjà émis restent donc valides.

4. Certificats de serveur et de client

Pour chaque entité, le demandeur génère sa propre clé P-256 et sa CSR, puis l'intermédiaire signe avec ext_serveur (SAN www.tp.local, tp.local) ou ext_client, avec -days 90. Le script pki.sh (étape 7) refait ces opérations et se termine par serveur.crt: OK, alice.crt: OK, bob.crt: OK .

5. Validations commentées

Commande	Résultat (fichier)	Interprétation
verify -verify_hostname www.tp.local serveur.crt	serveur.crt: OK (verif1)	Chaîne valide, nom conforme au SAN.
verify -verify_hostname autre.example serveur.crt	error 62 ... hostname mismatch (verif2)	Certificat valide, mais pas pour ce nom.
verify -purpose sslclient bob.crt	bob.crt: OK (verif3)	Usage client autorisé par clientAuth.
verify ex.crt (CN=www.example.com)	error 47 ... permitted subtree violation (verif_ex)	Nom hors du périmètre de l’intermédiaire.

Pourquoi l'AC a-t-elle pu signer le certificat www.example.com alors qu'il est refusé à la validation ? openssl ca signe ce qu'on lui demande et n'applique pas les contraintes de nom de son propre certificat. Ces contraintes sont contrôlées par celui qui valide la chaîne (le client). Le certificat existe, il est même inscrit dans index.txt, mais aucun client ne l'accepte. Les contraintes de nom limitent ainsi les dégâts si l'intermédiaire est mal utilisée ou compromise.

6. Révocation

J'ai supposé la clé d'Alice compromise : openssl ca -config inter.cnf -revoke alice.crt -crl_reason keyCompromise, puis génération des CRL de l'intermédiaire et de la racine, regroupées dans crls.pem. Observations :

• verify -crl_check_all ... rend error 23 at 0 depth lookup: certificate revoked (verif_alice_revoquee). L'option -crl_check_all vérifie la CRL de chaque niveau de la chaîne, d'où les deux CRL.

• crl_text.txt : émetteur Inter TP, Last Update 30 oct. 2026, Next Update 6 nov. 2026 (7 jours), numéro de CRL 4097 (0x1001), entrée de série 1001 révoquée le 27 oct. 2026, raison Key Compromise.

• index_inter.txt : la ligne d'Alice commence par R avec la date et la raison de révocation ; les autres lignes sont V. Y figurent aussi les certificats de test (www.example.com, court.tp.local) et le certificat de serveur renouvelé (série 1006).

7. Script pki.sh

pki.sh regroupe les étapes 2 à 4 (racine, intermédiaire, serveur, clients) et commence par set -euo pipefail : il s'arrête à la première commande en erreur. Comme il efface racine/ et inter/, je l'ai testé dans un dossier

séparé pour ne pas détruire la PKI utilisée au TP 3. L'exécution s'est terminée par trois OK et le message « PKI construite avec succès ».

8. Questions de réflexion

1. Pourquoi la clé de la racine doit-elle rester hors ligne ? Que faire si l'intermédiaire est compromise ? Et la racine ?

La racine signe peu de certificats et sa compromission détruit toute la confiance : on la garde donc hors ligne, hors de portée d'un attaquant réseau, et on ne s'en sert que pour signer l'intermédiaire et les CRL. Si l'intermédiaire est compromise, la racine la révoque (CRL de la racine), on crée une nouvelle intermédiaire avec une nouvelle clé et on réémet les certificats feuilles. Si la racine est compromise, il faut tout remplacer : nouvelle racine, nouvelle intermédiaire, nouveaux certificats, et redistribuer la nouvelle ancre de confiance à tous les clients, ce qui est lourd. C'est pourquoi on limite son usage.

2. À quoi sert pathlen:0 ? Quel risque si on l'oublie ?

Il interdit à l'intermédiaire de créer une AC en dessous d'elle : elle ne peut signer que des certificats feuilles. Sans lui, une intermédiaire compromise ou mal utilisée pourrait créer des sous-AC, ce qui étendrait l'attaque et compliquerait la révocation et l'audit de la chaîne.

3. Pourquoi la clé privée du serveur est-elle générée par le serveur et non par l'AC ?

Une clé privée qui ne quitte jamais son propriétaire ne peut ni être interceptée pendant le transport ni être conservée par l'AC. L'AC ne reçoit que la CSR (clé publique et nom) et signe. Cela garantit aussi que seul le serveur peut prouver son identité, ce qui est la base de l'authentification.

4. Le CN et le SAN contiennent tous deux www.tp.local. Lequel les clients modernes vérifient-ils ?

Le SAN. Le CN pour les noms d'hôte est abandonné depuis longtemps : les navigateurs et les bibliothèques actuelles ne regardent que le SAN. C'est d'ailleurs ce que montre verify -verify_hostname, qui compare le nom aux entrées DNS: du SAN.

5. Quelles sont les limites des CRL ? Comparer avec OCSP et les certificats de très courte durée.

Une CRL est publiée périodiquement : entre deux publications, un certificat révoqué est encore accepté ; elle peut devenir volumineuse ; si elle expire sans être renouvelée, un client strict rejette tous les certificats (risque réel avec default_crl_days = 7). OCSP interroge l'état d'un certificat en temps réel, mais impose un serveur toujours disponible, révèle au serveur OCSP les sites visités et pose un dilemme en cas d'échec de la requête ; l'agrafage OCSP atténue ces défauts. Les certificats de très courte durée réduisent le problème à la source : un certificat volé expire vite, la révocation devient presque inutile, au prix d'une automatisation indispensable du renouvellement.

6. Politique de certification minimale (dix lignes).

1. Qui peut demander : les administrateurs système du domaine tp.local pour les serveurs ; les utilisateurs enregistrés pour les certificats clients.
2. Identité vérifiée : le demandeur prouve qu'il contrôle le nom demandé et l'identité est confirmée par le responsable du service.
3. Clé privée générée par le demandeur, jamais transmise ; la CSR seule parvient à l'AC.
4. Seuls les noms sous tp.local sont émis ; toute autre demande est refusée.
5. Durée des certificats feuilles : 90 jours au maximum ; intermédiaire 5 ans ; racine 10 ans.
6. Algorithmes : courbes P-256 (feuilles, intermédiaire) et P-384 (racine), signatures SHA-256 ou SHA-384.
7. La clé de la racine est hors ligne ; la clé de l'intermédiaire est protégée (droits 600).
8. Révocation sur demande du titulaire ou sur suspicion de compromission, dans la journée, avec la raison.
9. La CRL est republiée au moins tous les 7 jours et avant son expiration.
10. Toute émission et révocation est tracée dans la base index.txt, conservée et sauvegardée.